

4.1 ToolTypes

4.1 Tool Types

Debugger

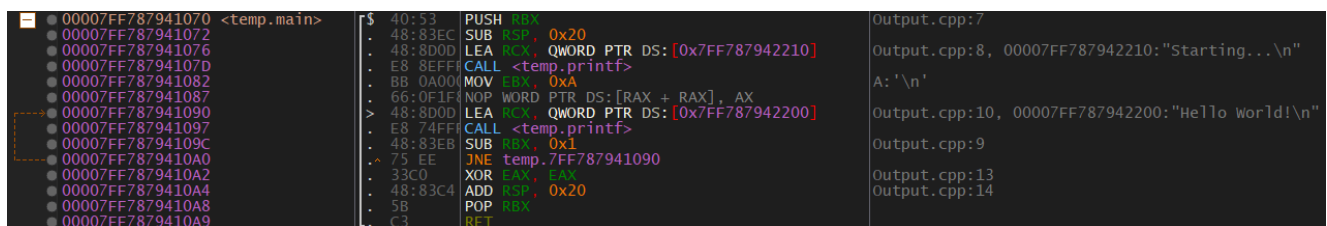
You've probably heard of a debugger before. It allows a program to step through their code and analyze what it's doing line-by-line. We will be using these ourselves. Unfortunately for us, we won't have the source code, instead we will be looking at Assembly.

A crash course on debugging can be found in [4.2 Debugging](#).

Both Visual Studio and x64dbg have debuggers built into them.

Disassembler

A disassembler will take a binary and present it's Assembly code. Disassemblers are the backbone of reverse engineering. Here is the disassembly of a small function:



The screenshot shows a debugger window with three panes. The left pane shows memory addresses from 00007FF787941070 to 00007FF7879410A9, with the label <temp.main>. The middle pane shows the assembly code for these addresses, including instructions like PUSH, SUB, LEA, CALL, MOV, NOP, JNE, XOR, ADD, POP, and RET. The right pane shows the corresponding source code from Output.cpp, including lines 7, 8, 9, 10, 13, and 14, which contain printf statements for 'Starting...', 'A:', and 'Hello world!'.

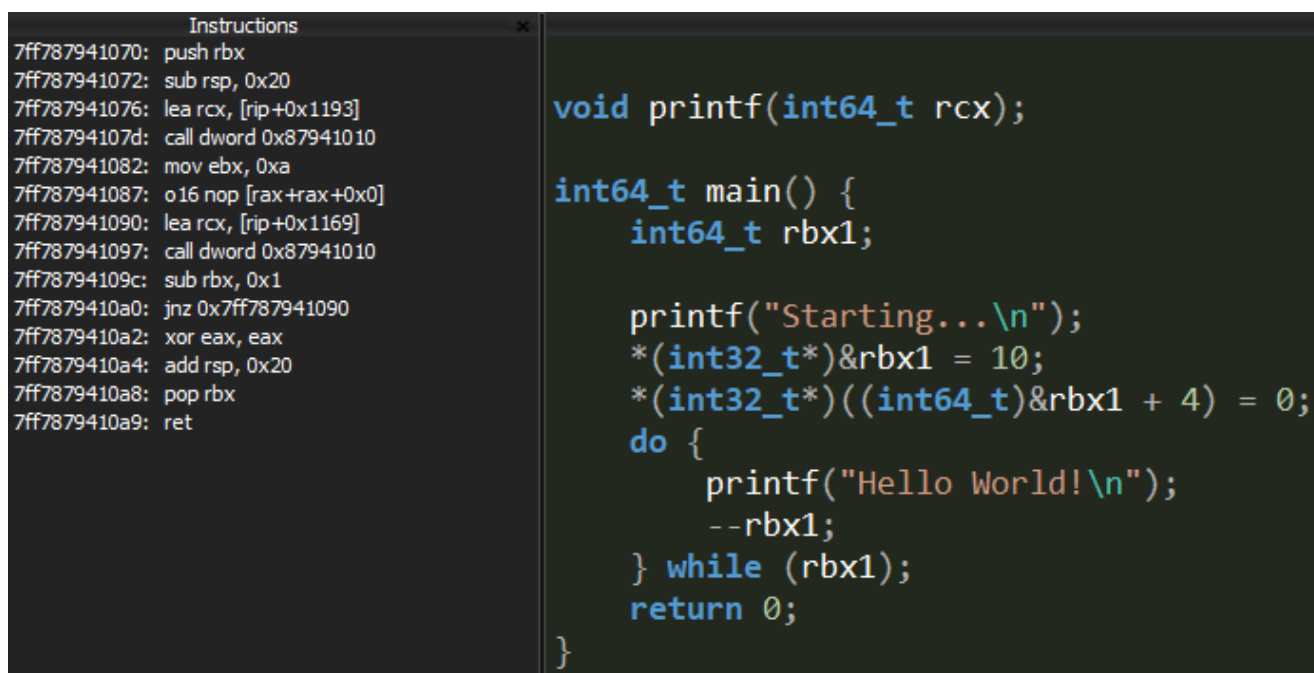
Ghidra and x64dbg can disassemble a program. Visual Studio also allows you to see the disassembled version of your code. This can be extremely helpful for learning or testing. If you want to enable this you can do so by setting a breakpoint in the program then going to Debug > Windows > Disassembly.

You can view the memory by going to Debug > Windows > Memory > Memory #

Decompiler

A decompiler attempts to take a compiled program and turn it back into the original source code. These are often inaccurate but they are useful for identifying parameters, loops, conditionals, and sometimes structures. They can also be a good source to *help* identify data types (never trust decompiler data types, only use them as additional help).

Here is the decompiled version of the function shown in the disassembler portion:



```
Instructions
7ff787941070: push rbx
7ff787941072: sub rsp, 0x20
7ff787941076: lea rcx, [rip+0x1193]
7ff78794107d: call dword 0x87941010
7ff787941082: mov ebx, 0xa
7ff787941087: o16 nop [rax+rax+0x0]
7ff787941090: lea rcx, [rip+0x1169]
7ff787941097: call dword 0x87941010
7ff78794109c: sub rbx, 0x1
7ff7879410a0: jnz 0x7ff787941090
7ff7879410a2: xor eax, eax
7ff7879410a4: add rsp, 0x20
7ff7879410a8: pop rbx
7ff7879410a9: ret

void printf(int64_t rcx);

int64_t main() {
    int64_t rbx1;

    printf("Starting...\n");
    *(int32_t*)&rbx1 = 10;
    *(int32_t*)((int64_t)&rbx1 + 4) = 0;
    do {
        printf("Hello World!\n");
        --rbx1;
    } while (rbx1);
    return 0;
}
```

Ghidra and x64dbg both have decompiler. Ghidra has one built-in and x64dbg has a plugin called "Snowman" that does it.

Viewers

There are different ways to represent a program or parts of a program. Hex viewers and editors will show you the program represented in hexadecimal. There are even programs that will try to view the program as ASCII.

Utilities

There are various other tools that we can use to gather more information about a binary. For example, DUMPBIN can be used to look at imports, exports, headers, symbols, and more.

The best way to figure out all of these tools is to use them.

Static vs Dynamic Analysis

- **Static Analysis** - Analyzing the binary when it's not running. In other words, viewing the binary as it is on disk. Ghidra is a tool mainly used for static analysis
- **Dynamic Analysis** - Analyzing the binary as it's running in memory. We mainly use x64dbg for dynamic analysis

I also consider reversing a DLL in x64dbg using DLLLoader to be static analysis. This is because we can't step through the exported functions (they are never called).

DLLLoader refers to a simple program (.exe) that loads a DLL into memory (RAM) so dynamic tools like x64dbg can attach to the DLL and analyse it.

[<- Previous Lesson](#)

[Next Lesson ->](#)

